J.BURROWS

A4 Notebook

70gsm paper

8mm ruled

120 pages

FSC MIX Paper FSC® C011666

MADE IN CHINA
OFFICEWORKS LTD
250-262 EAST BOUNDARY RD
EAST BENTLEIGH VIC 3165 AUSTRALIA

9 341694 517599

JBA4PPNBGY 07/19 GF

Minimum structure of a c program

```
// This is a comment.                          authorship declaration
/* This is another                             *contribute to marks
   comment.          */

#include <stdio.h>                             include standard library
                                               std: standard io: input output
int
main(int argc, char *argv[]) {

    // ... codes

    return 0;

}
```

Basic function

① `printf("\n");` ← e.g `printf("a=%d", a);`

* 在C语言里，"" 和 '' 不一样，单引号不能代替双引号

② `scanf("%f", &radius);` ← 直接用在printf("")后，如 printf("Enter no:")

e.g `scanf("%d %lf %c", &a, &b, &c);`

Example:

```
int x;
printf("Give me a number: ");
scanf("%d", &x);
```

Define

Define Constants

```
#define  MIN_PER_HOUR  60

#define  ERROR_MSG  "Values must not ..."
```
↓ 句尾不需要 semicolon

Define variables

```
int  sum = 0;
```

* variable must be declared before referencing

```
float  a, b, c;

double  x, y, z;
```

Data type

integer / decimal (base10)	%d	integer	%10d ← 10个字符位
	%f	float and double	%8.2f ← 共8个字符位, 保留小数点后两位
	%e	exponential form e.g. 6.67e-11	
	%c	char	%-50c ← 自动右对齐
	%s	string	
long float	%lf	double ← only for reading 'double'	
		↓ always prefer as it has higher precison	

* when using printf, it is okay to use %f to print a double
↓ not in scanf

Operators

		Expression	Result
*	multiplication	5.0 * 3.0	15.0
		5 * 3.0	15.0
		5 * 3	15
/	division	19.0 / 4.0	4.75
		19.0 / 4	4.75
		19 / 4	4
%	remainder after division	19.0 % 4.0	illegal
		19.0 % 4	illegal
		19 % 4	3
+	addition	7.5 + 3.0	10.5
		7.5 + 3	10.5
		7 + 3	10
-	negation	-3	-3
		-5 * -3	15
-	subtraction	7.0 - 13.2	-6.2
		7 - 13.2	-6.2
		7 - 13	-6

Grok - compile the file using terminal

Type：第一行 `gcc -Wall -std=c11 -o program program.c`

- gcc：program arguments
- -Wall："show all warning"（W要大写）
- -std=c11：compile using the C11 language (2011)
- -o program：output the program to the file, "program"
  - the program we execute with `./program`
  - in windows 这台是 .exe
- program.c：the input source code

第二行 `./program`

*每打完一行，按一次 enter

Terminal commands

pwd　tells you which directory (folder) you are in

ls　lists all files in your current directory

more and cat　show the content of given file.

Integer format

left aligned integer with width 6　`printf("%-6d", 12345);`

"zero-filled" integer with width 6　`printf("%06d", 12345);`

Floating point format

pi = 3.14159265

`printf("%f", pi);`	=>	"3.141593"
`printf("%.1f", pi);`	=>	"3.1"
`printf("%.0f", pi);`	=>	"3"
`printf("%8.0f", pi);`	=>	"　　3"
`printf("%08.0f", pi);`	=>	"00000003"
`printf("%.10f", pi);`	=>	"3.1415926500"
`printf("%e", pi);`	=>	"3.141593e+00"
`printf("%.2e", pi)`	=>	"3.14e+00"

Chapter 2 特殊符号表 Escape characters

char	meaning
\n	newline
\\	backslash \
\t	horizontal tab
\r	carriage return
\'	quotation mark '单引号
\"	double quotation mark " 双引号
%%	percent %

Convert data type (casting)

→ Float to ~~dobl~~ double

```
float x = 1.2
printf ("%lf", (double)x);
```

或

```
average = (double) (num1 + num2) / 2 ;
```

/ / → Implicit casting: `double y = x;`

Name convention in C for identifier/variable

★ Any valid C identifier must begin with a letter or underscore(_), then can use any combination of digits, letters and underscores.

★ Convention table

Case type	Example	when to use in C
Snake Case (popular in Python, C)	an_awesome_name	most common
Camel Case (popular in Java)	anAwesomeName	less common
Upper Case Snake Case	SOME_AWESOME_NAME	for constants

Chapter 3 Relational and logical operators

Operator	Operation
<	less than
<=	less than or equal
>	greater than
>=	greater than or equal
==	equal to
!=	not equal to
&&	and
||	or
!	not

★ Relational and logical expression generate int values that can be interpreted and used as if they correspond to true and false

★ and && : 第一个数只有在第一个数不为0时才会被evaluate

```
0 && 0  ⇒ 0
0 && 1  ⇒ 0
1 && 0  ⇒ 0
1 && 1  ⇒ 1
```

or || : 第一个数只有在第一个数为0时才会被evaluate

```
0 || 0  ⇒ 0
0 || 1  ⇒ 1
1 || 1  ⇒ 1
```

★ 其他: !(e1 && e2) has the same truth value as (!e1 || !e2)

and !(e1 || e2) has the same truth value as (!e1 && !e2)

Precedence of operators

Operators	Operation class	Precedence
++ , --	post-increment, pose-decrement	Highest
! , - , (type)	unary not, unary negation, typecasting	
* , / , %	multiplication	
+ , -	addition	
< , > , <= , >=	comparison	
== , !=	equality	
&&	and	
||	or	
= , += , *= , etc	assignment	Lowest

IF Statement

```
if (guard) {
    ...
} else if (guard) {
    ...
} else {
    ...
}
```

→ Non-zero is interpreted as being 'true'; Non-zero is interpreted as being 'false'; There is no boolean in c language.

→ condition / guard must enclose by brackets.

Exit function

✓ Exit silently: exit (EXIT_SUCCESS)

✗ Exit and signal error: exit (EXIT_FAILURE)

Example:

```
if (scanf ("%d%d", &x, &y) != 3 ) {
    printf (" invalid input for integer");
    exit (EXIT_FAILURE);
}
```

SWITCH statements (avoid using it)

- General form:

```
switch (integer expression) {
case (integer1) :
    statement 1 ;
    break ;
case (integer2):
case (integer3):
    statement 2
    break ;
default :
    statement 3
    break;
```

两个case有一样的处理方式

田 The integer expression is first evaluated, the result will be checked against the case. If not matched with any case, then default case is reached

✳ Extra notes: strings can not be added together in C Like Python but we can do printf ("Hello" "World);

For Loop

```
for (initialize ; guard ; update) {
    statement
}
```

- Example:

```
for (divisor = 2; divisor*divisor <= n ; divisor++) {
    if (n% divisor == 0) {
        isprime = 0;
        break;                    //break from Loop
    }
}
```

While Loop

```
while (guard)
    statement
```

- # Read any number of input, and find maximum value

```
while (scanf("%d", &next) == 1) {
    if (next > max) {
        max = next;
    }
    n += 1;
}
```

- # Read until it is End Of File (EOF) → ctrl + d

```
while ((input = getchar()) != EOF) {
    ...
}
```

The do statement

```
do
    statement
while (guard)
```

Break; VS Continue;

- break; immediately stops iteration, terminating the loop as if the guard returned false.
- continue; skips the current iteration, but continue looping. For that loop, the rest of the body is skipped, and we return to the update (for) or guards (while) part of the loop.
- Both break and continue would only affect inner-most loop in nested loop

i++; VS ++i;

- i++ will evaluate to the value of i, then increment i
  延后性，下次出现时才会是 increase 完后的 value.
- ++i will increment i, then evaluate the value
  即时性

Define 的无脑复制性

- Example:

```
#define MAX_PETS 2+3
...
printf("%d", MAX_PETS *3);
```

=> 11
↑ 因为被 evaluate 成 5*3+2 了

Getchar() 和 Putchar()

示例：

```
char c;
printf("Enter character: ");
c = ~~getchar~~ getchar();

printf("Character entered: ");
putchar(c);

return(0);
```

Output

```
Enter character: a
Character entered: a
```

chap5

Function

→ A function consists of: • a declaration for the return and argument types
• local variable declarations
• body
• usually at least one return statement.

→ Example of a function

```
int
isprime (int n , double n1) {
    ...

    return 1 ;
}
```

→ Properties of function

- A function can only return a single value and all local variables and argument variables are discarded at the time the function returns.
- The declared return and argument type must 一樣的
- we don't need to declare a function before we define / reference it. [But 通常放 #define 下 和 main 上]

Useful function from useful library

Library	Function
<ctype.h>	Each of following takes an int argument, return 1 or 0 isalpha() , isascii(), isdigit() islower() , isspace(), isupper() toupper(c) , tolower ↑ return uppercase version of character c
<math.h>	M_PI (value of π) M_E (value of e) M_SQRT2 (value of √2) sqrt(double x) ... ...

Recursive function

```
// calculate the n'th triangle number using recursion

int
t_rec (int n) {
    if (n == 0) {                    ] Base case
        return 0;
    } else {                         ] Recursive case
        return n + t_rec(n-1);
    }
}
```

Void Returning function (AKA 'procedure')

- Example:

```
void happy() {
    printf(" :) \n");
}
```

Assert

- assert is a function in the assert.h library
  Any condition given to assert must hold true, otherwise the program will exit with an error

- Example:

```
# include <assert.h>
assert( scanf("%lf", &n) == 1 );
```

chapter 6

Pointer

- Pointers are variables that store memory addresses.
- int *p creates an int pointer, p
- &x means address of x
- int *p = &x; Make int point p point to x
  OR assign int pointer p to the address of x
- *p on its own, access the value p is pointing to.
- print memory address %p printf("&x = %p", &x);

* Do not use Global variable

Static variable

- A static variable is local to the function it is declared in, but remains its value between calls to that functions.
  被存着但只在 local function 里存着
- Example:

```
int
main(int argc, char *argv[]) {
    int x = 3, z = 5;
    printf("main: x = %2d, z = %2d \n", x, z);
    func(x);
    printf("main: x = %2d, z = %2d \n", x, z);
    func(z);
}

void func(int x) {
    static int z = 7;
    x = x + 1;
    z = x + z + 1;
    printf("func: x = %d, z = %d \n", x, z);
}
```

main: x=3, z=5
func: x=4, z=12
main: x=3, z=5
func: x=6, z=19

Property of pointer

- pointers do not have an initial value assigned, they must be given a value before they can be used.
- the address-of & and dereferencing operators * are both unary operators, and hence, they have high precedence

Use of pointer - change ~~本体~~ value

```
int   n = 11 , *pi ;
pi = &n ;
print (" *pi = %d ", *pi);
*pi = *pi + 1 ;
printf (" n = %d , *pi = %d ", n, *pi );
```

⇒ *pi = 11
n = 12 , *pi = 12

Use of pointer - as argument in a function

```
# SWAP TWO VARIABLES.
void
intswap ( int *a , int *b ) {
        int temp = *a ;
        *a = *b ;
        *b = temp ;
}

int
main ---- {
    int  x1 = 5 , x2 = 6 ;
    intswap (&x1, &x2);
    printf (" x1= %d, x2 = %d ", x1, x2);
}
```

x1 = 6, x2 = 5

☆ Must do : When a function wants to alter its arguments, it should be designed to receive pointers, and when it is called, pointers must be passed!

Chap 7

Array

- Array store a number of elements of the same type in a continuous block of memory.
- Array is zero-indexed
- 和 python 不同处
  - → array cannot use negative indices
  - → array cannot be sliced, can't do A[1:3]
  - → array cannot be resized, it must have a positive size, and size cannot be changed after been declared.
  - → array cannot be compared with another array's as array variables are pointers. When do A == B, it only check if &A[0] == &B[0].
  - → But we can do B = A as B is a pointer
    int A[] = {1, 2};
    int *B = A;
  - → Array can't be copied with A=B

Declare an array

其一：int numbers[10];　| an array that stores 10 integers

其二：#让用户决定array的大小

```
int n;
scanf("%d\n", &n);
int arr[n];
```

Initialise arrays

① int numbers[10] = {1, 2, 3, 4}
↳ 其他后面的数 is automatically initialised to 0

② int numbers[200] = {0}
↳ 第一位数为0，其他后面的数自动为0.

③ int numbers[] = {42, 43, 1, 2, 3}
↳ 聪明小C能算出array大小为5，所以不用填

Use array in function

理论知识

- When we passing an array as an argument to a function, we are passing only the address of the first element of the array. We are not copying the entire array to the function. Hence, modifying an array in a function will modify the original array.

实操

- void print_array (int A[], int n);
  ↳ takes an int array of length n.
- void print_array (int A[5]);
  ↳ fixed length
- 不对的例子 void print_array (int A[n]);
  ↳ 没有 declare n 是什么
- call function: print_array (A, 8)

• # add items /value to array 填充array

```
int A[5], i;
for (i=0; i<5; i++) {
    A[i] = i+2;
}
```

Library of array <arrayops.h> ← only for chap 7 exercise

Function	功能
int read_int_array (int A[], int n)	read integers from input and store them in an array
void int_swap (int *p1, int *p2)	exchange the values of two variables
void sort_int_array (int A[], int n)	sort n elements of array A in increasing order (2, 3, 4, 5)
void print_int_array (int A[], int n)	print the elements of an array on one line

Two-dimensional array

Create two-dimensional array

① `int (A[5])[10];`

②
```
#define ROWS 5
#define COLS 10
int A[ROWS][COLS];
```

③ `int mat[][4] = {1,1,1,1,2,2,2,2}`
↳聪明小C能算出 first dimension是2

④ `int board[2][3] = {{1,1,1}, {2,2,2}};`

⑤ 错误方法Illegal: `int board[][] = {1,1,2,2};` (除第一个dimension外不能空)
↳ It's not clear what board[1][0] is

☆ Dimensions must be given for all but the first dimension.

Typedef

- The typedef declaration defines a new type. It can be an alias for a simple type, or a compound type.
- Type defs should be used for all important recurring types in a program.
- typedef double vector_t [SIZE];
  is telling compiler that "everytime you see the variablename vector_t used as a type in a declaration, it means an array of double containing SIZE values.
- 实操

```
typedef double vector_t [SIZE];

void
vector_add (vector_t A, vector_t B, vector_t C, int n){
    int i;
    for (i=0; i<n; i++) {
        C[i] = A[i] + B[i];
    }
}
```

Array of characters

- 定义:
  - C imposes a requirement on strings when declaring. It stipulates(规定) the last element as the character constant '\0'. The null byte acts as a sentinel.哨兵

☆实操 Declare

方法
1. char s1[5] = {'H', 'e', 'l', 'l', 'o'};
2. char s2[6] = {'W', 'o', 'r', 'l', 'd', '\0'};
3. char s3[100] = "Goodbye";
4. char s4[] = "Pluto";
5. char *s5 = "Farewell Neptune";

☆ Library of string <strings.h>

→ int strlen(char *s)
Return the number of characters in s, not including the null byte.

→ ... more function see textbook pg 127.

☆ String literals

- String literals are not strings; String literals are read-only, they are stored in memory in the code part of the program instead of the stack or heap. They cannot be modified 有点像 pointer
- char str[] = "Nice";
  char *p = str;
  *p = 'D';
  ↓ 指向str[]第一位 char
  现在 str[] 和 *p 都表示 "Dice"
  因为 *p point to str[]

chap 12

Big-Oh notation: O (g(n))

the big-Oh notation allows the growth rate of functions to be compared in a way that is independent of their numeric value.

Quick sort: Quick sort splits the input array into three sections by partitioning(划分) the elements relative to a pivot value; and then recursively sorts two of the partitions.

chap 8

Structure

Format

```
typedef struct {
        double distance
        double radius
} one_planet;
        ↳ name of the struct variable
```

Definition: structure is a way to group related variables into one place. Structure 像一套组合拳，里面可以有不同 data type 的数据

Initialise

Method 1

```
one_planet.distance = 149.6;
strncpy (one_planet.name, "Earth", PlanetStrlen);
                ↓ destination      ↓ source.
```

Metho 2

```
one_planet = {149.6, 8.5};
```

将 Typedef 与 Structure 组合起来

Declaration of a structure using a typedef

```
typedef char pstr_t [PLANTSTRLEN+1]

typedef struct {
        pstr_t name, ~~object~~ orbits;
        double distance;        /* million km */
        double mass;            /* kilogram */
        double radius;
} planet_t;

planet_t the_earth = {"Earth", "Sun", 149.6, 800, 8};
planet_t the_moon={"Moon", "Earth", 0.38, 700, 17};
```

每个星球都有自己的信息。

Properties of Structure

1. Assignment of structure B = A is possible as the exact type and size of any array within the ~~stu~~struct always known

2. Comparison of structures B == A is not possible
   If must compare, must compare field by field.

3. Printing of ~~str~~ the whole struct is not possible,
   Must print component in structure one by one.

4. Local structure is short-lived, hence, don't return a pointer to a local structure

Print components in Structure

```
Printf ("\t mass is %.1f kg", one_planet.mass);
```

⇒ ⟵ 4格 (a tab) mass is 8.0 kg

Read in data for a struct variable

Method 1;

```
planet_t new_planet;
printf ("Enter data : \n");
scanf ("%s %s %lf %lf %lf",
        new_planet.name,
        new_planet.orbits,
        &new_planet.distance,
        &new_planet.mass,
        &new_planet.radius );
```

Method 2;

```
int read_planet_ptr (planet_t *planet) {
    int nvals_read;
    printf ("Enter data : \n");
    nvals_read = scanf ("%s %s %lf %lf %lf",
                planet -> name,
                planet -> orbits
                planet -> distance
                planet -> mass
                planet -> radius);
    if (nvals_read != 5) {
            return EOF;
    } else { return 0; }
}
```

Nested struct

```
typedef char namestr[NAMESTRLEN+1];
                    ↳ 提前设好的 limit

typedef struct {
    namestr given, middle, family;
} fullname_t;

typedef struct {
    int yy, mm, dd;
} date_t

type def struct {
    fullname_t name;
    int studentid;
    date_t dob;
} student_t;
```

Pass struct as argument into function

→ Structures are passed into functions in the same way as scalar arguments are. 例子在上一页的 method 之一 例子 pass 的是 pointer.

→ For most purposes, it is more appropriate to pass a structure pointer ~~also~~ rather a structure. Passing a pointer allows the function to alter the components in the underlying compound variable.

→ 其他, when array of structure is created, it is easier to pass pointer.
E.g read_planet_ptr (planets+i) passes a pointer to the ith element of planets and read values into the five variable that make up planets[i].

→ Call a function that need structure as argument

```
planet_t planet;
if (read_planet_ptr(&planet) != EOF)
    {
    ...
}
```

→ Return structure function

```
planet_t
read_planet (void) {
    planet_t new_planet;
    ...
    return new_planet;
}
```

Avoid overflow in structure

☆ A variable is used to count the number of things stored in the structure (e.g int nplanets)

☆ Since the size / number of things in a structure is useful to know when accessing the structure, hence, it is reasonable to combine them together. For example:

```
typedef struct {
    int nplanets;
    planet_t planets[MAXBODIES];
} solar_system_t;
```

declare: solar_system_t solar_system;
a new struc

Select thing in structure

- '[]', '.', 和 '->' 有相同的优先级
- Example:

```
planets[0].orbits
planets[0].orbits[3]
video.created.yyyy
v -> released.dd
```

都是从左往右执行的

Compare two structures

- a comparison function should always be written as something like:

```
int
compare_video_label (video_t *v1, video_t *v2){
        if (v1.lec_num < v2.lec_num) return -1;
        if (v1.lec_num > v2.lec_num) return +1;
        if (v1.lec_seq < v2.lec_seq) return -1;
        if (v1.lec_seq > v2.lec_seq) return +1;
        return 0;
}
```

chapter 10

Dynamic array: dynamic arrays allow runtime memory to be efficiently used, as programs can adjust their memory consumption to the scale of the input data.

Control allocation of memory

以下 function 都是在 <stdlib.h> 里

Functions	Call the function	Purpose
size_t	sizeof (thing)	Returns the number of bytes required to store the type or variable supplied by argument thing. Not really a function
void	*malloc (size_t size)	Allocates a fresh segment of memory containing n bytes (size is an integer)
void	*calloc (size_t nmemb, size_t size)	Allocates a fresh segment of memory that contains the number of bytes required to store an array. The array has nmemb items, each requires size bytes. Once allocated, the memory segment is set to byte zeros
void	*realloc (void *ptr, size_t size)	Allocates a fresh segment of memory that has size bytes, then copy the contents in the memory segment (start from ptr (pointer)) into the fresh segment and then call free(ptr)

continue →

void free(void *ptr) Deallocates the segment of memory indicated by ptr and returns it to the pool of available memory.

展开讲讲

sizeof (thing)

- The value returned by sizeof is of type size_t, it is essentially an unsigned integer.
- 尺寸表

data type	bytes
char	1
int	4
float	4
double	8

- sizeof() distinguish between arrays and pointers

Initialise : double A[10]
char *p = "mary merry me"

size of (A) is 80
sizeof (*A) is 8
size of (p) is 8
size of (*p) is 1

} 两个都是pointer, pointer has size 8

- Example call : sizeof ('A')

memory allocation

void *malloc (size_t size)

- Function malloc returns a pointer to a segment of memory of ~~require~~ requested size in bytes, OR null if no such segment can be allocated.
- All allocated memory is automaticall returned to the system when a program terminate

- example call:

all_words [numdistinct] = (char *) malloc (1 + strlen (one_word));

这个array [numdistinct] 之后自己数据都被存在新来的 memory 里

assign the pointer type to be char

return a void pointer

when using malloc to create an array to store a string, one extra character must be requested, to hold the terminating null byte.

void * realloc (void *ptr, size_t size)

- The function realloc is used to manage arrays that must expand to accept more data. The first array must be created with malloc, thereafter array size should grow as a geometric sequence

void free (void *ptr)

- Each call to malloc should be matched by a later call to free() 占用完 memory 后，要释放 memory

- example call

staff_t *sara;
free (sara);

Exit if no memory can be used

Function 1: exit_if_null

- this function test each pointer after any of the memory allocation has been made.

```
void
exit_if_null (void *ptr, char *msg) {
        if (!ptr) {
                printf(" unexpected null pointer\n");
                exit (EXIT_FAILURE);
        }
}
```

* Null is equivalent to integer zero, meaning !null will evaluate to 1

Function 2: assert in <assert.h>

- assert (all_words [numdistinct] != null);
- assert (0 ≤ numdistinct && numdistinct < current_size);
  this one checks that two variables are maintaining an expected relationship.

Linked structures

- while malloc can create / allocate space for an array, it is also possible to use malloc to allocate more space for a single variable, or for a structure
- Example.

```
staff_t *sara;
sara = (staff_t *) malloc (sizeof (*sara));
```

make the returned void pointer to have type staff_t

Recursive data structure

→ Declaration

```
typedef struct node node_t;
```

有流程 typedef struct node { } node_t;

```
struct node {
    data_t data;      → the information that is to be stored in each node, can be in any type (int, char, staff_t)
    node_t *next;
};
```

// node_t is a structure in which one of its components is a pointer to an object of type node_t. Which means one of the components of that second structure is a pointer to a third, and so on.

// node_t is a Recursive data structure.

// In the case of node_t, the base case is a next pointer that has the value Null (pointer 指无可指)

// Using these node_t variables, it is possible to build a chain, or linked list, with each node_t in the chain using its next pointer to indicate the next element.

⇒ The first pointer in the chain must be a variable in the program, but thereafter, the nodes in chain can be created through malloc

→ Linked list

- A linked list is a one-dimensional data structure in which objects are threaded together using a pointer in each node.

- typedef struct {
    node_t *head; → indicate the first item in linked list
    node_t *foot;
  } list_t;

⇒ if the linked lisk is empty, then both pointers (*head and *foot) are NULL

→ Linked list's four main operations

1. insert prior to the first item
2. append after the last item
3. fetch the contents of the first item
4. delete the first item

// insert prior to the first item

```
list_t *insert_at_head (list_t *list, data_t value) {
    node_t *new;
    new = (node_t *) malloc (sizeof (*new));
    assert (list != NULL && new != NULL);
    new -> data = value;
    new -> next = list -> head;
    list -> head = new;
    if (list -> foot == NULL) {
        list -> foot = new;
    }
    return list;
}
```

如果new已经是第一个node，则其data应存着value.
其point to 的next point应是现在的第一个node

if this is the first insertion into the list.

// insert item/data at the end of a list of data

```
list_t
*insert_at_foot (list_t *list, data_t value) {
    node_t *new;
    new = (node_t *) malloc (sizeof (*new));
    assert (list != NULL && new != NULL);
    new -> data = value;
    new -> next = NULL;
    if (list -> foot == NULL) {
        // this is the first insertion into the list
        list -> head = list -> foot = new;
    } else {
        list -> foot -> next = new;
        list -> foot = new;
    }
    return list;
}
```

如果new是最后一个node则其data应存value

其point to 的next pointer 应是NULL

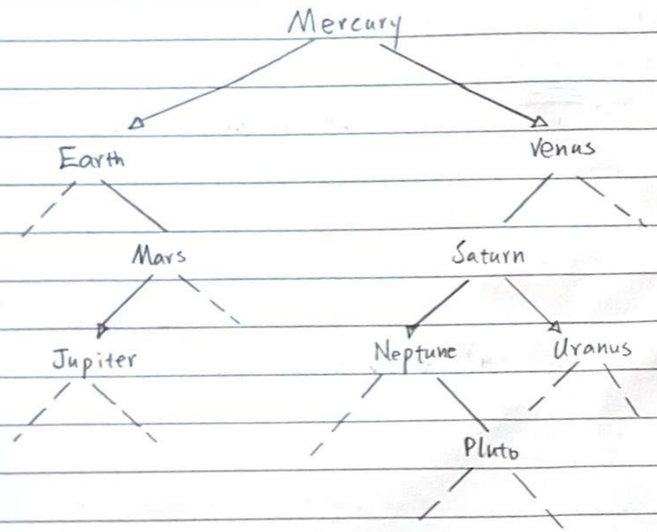
A stack: a data structure in which the most recently inserted is returned next. 先进的data先出

A queue: a data structure in which the least recently inserted item is returned next. 最早进的data先出

Binary search trees

★ A binary tree is a two-dimensional data structure in which objects are threaded together using two pointers in each node.

★ 图示

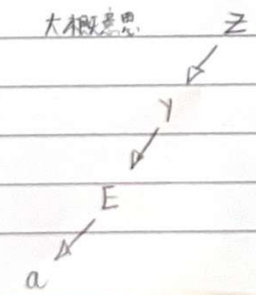


→ insertion order 和 distance from the sun

★ Speed of binary search

- Over all possible initial permutations of n items, the average depth (comparison needed to find a word) is about $1.4 * \log_e(n)$
  Hence, average search cost will be $O(\log_e(n))$ key comparisons, whether successful or unsuccessful.
- In worst case, tree is a stick, and search will take $O(n)$ key comparisons.

大概意思



Function pointers

◎ Declaration and use of a function pointer

```
double (*F)(double);
```

a pointer that takes a 'double' variable as argument and return a double back.
↓ can have two argument
Eg. double (*F)(double, int);

```
F = sqrt;
```

can be assigned because sqrt() in <math.h> is a function that takes a 'double' variable as argument and return a double.

```
printf("%lf is F(x)", F(x));
```
↳ x is predefined as 2

=> 1.41 is F(x)

◎ Passing function pointer as argument

// define a function

```
int
double_ascending (void *v1, void *v2) {
    double *d1 = v1, *d2 = v2; // compare two double variables
    if (*d1 < *d2)  return -1;
    if (*d1 > *d2)  return +1;
    return 0;
}
```

continued →

// define a function that takes a function pointer as
// argument.

```
int
is_sorted (void *A, size_t nelem, size_t size,
           int (*cmp) (void *, void *)) {
    char *Ap = (char *) A    // array in void type convert to char type
    while (nelem > 1) {      // int and char... take different no of bytes.
        if (cmp (Ap, Ap + size) > 0) {
            /* not in order */
            return 0;
        }
        Ap += size;
        nelem -= 1;
    }
    /* all elements are in order */
    return 1;
}
```

// Whether ~~Sort~~ an array of double is sorted using function pointer

```
int
main (int argc, char *argv[]) {
    double X[] = {1.81, 3.43, 7.64, 7.68, 8.16, 9.86};

    if (is_sorted (X, sizeof(X)/sizeof(*X),
                   sizeof(*X), double_ascending)) {
        printf("Array X is ascending\n");
    }
}
```

function name itself is a pointer, just like array name X itself is a pointer

Sort array of any type (polymorphic 多态)

- qsort : a library function described by the <stdlib.h> header file.

// a comparison function the return true if string1 ('a') < string2 ('banana')

```
int
string_ascending (const void *v1, const void *v2) {
    return strcmp (*(char **) v1, *(char **)v2);
}
```

// sort an array of string ascendingly

```
...
char *S[] = {"koala", "wombat", "kangaroo"};
qsort (S, sizeof(S)/sizeof(*S), sizeof(*S),
       string_ascending);
```

// The qsort function is polymorphic (多态), and can be used to sort any array, provided a suitable comparison function is written.

Chapter 11

File Manipulation

Three standard files

- Below three files are always opened when a program is executing.

1. stdin, for input from the keyboard, and available for redirection by the shell.

2. stdout, for output to the screen, and arailable for redirection by the shell

3. stderr, output to the screen, and available for separate redirection by the shell.

- In a program, printf(...) is a call to fprint(stdout...) scanf(...) just calls fscanf(stdin, ...)

Functions that help us manage files [all in <stdio.h>]

Function	Purpose
FILE *fopen(char *fname, char *mode) (return a file pointer.)	Returns a pointer to a file that has been made ready for the specified operation, or NULL is returned if the file does not exist. 'fname' is the name of the file to be opened; 'mode' describes the operations. "r" for reading, "w" for writing, "a" for appending. In mode "w", if the file has content already, it will be cleared. In mode "a", if the file has content already, new content will be written at the end of the file.
FILE *freopen(char *fname, char *mode, FILE *~~stream~~ stream)	closes the file specified by stream, and then attempts to re-open it using the first two arguments.
int fclose(FILE *stream)	flushes all pending output to file stream and then close it.
int getc(FILE *stream)	Reads a single character from stream, like getchar. Returns EOF if end of file is detected
int putc(int ch, FILE *stream)	Write character ch to file stream, like putchar. Returns EOF if the output operation fails.

size_t fread (const void *A, size_t size, size_t nelem, FILE *stream)

Reads binary data from stream
A total of nelem objects each of size bytes
are sought ; the number of such objects
actually read is returned.

size_t fwrite (const void *A, size_t size, size_t nelem, FILE *stream)

Writes binary data from A to the file stream
A total of nelem objects each of size bytes
is to be output ; the number of such objects
actually written is returned.

size_t fseek (FILE *stream, long offset, int whence)

Moves the current location within file stream
by offset bytes relative to the starting point
specified by whence

fscanf and fprint

- The function
  internal
  characte
- ASC
  rep

Function Purpose

int fscanf (FILE *stream, const char *format, ...)
Input from file stream, controlled by format, like scanf. Returns the number of values read.

int fprintf (FILE *stream, const char *format, ...)
Output to file stream, controlled by format, like printf. Returns the number of bytes written

size_t fread (const void *A, size_t size, size_t nelem, FILE *stream)
Reads binary data from stream to address A. A total of nelem objects each of size bytes are sought; the number of such objects actually read is returned.

size_t fwrite (const void *A, size_t size, size_t nelem, FILE *stream)
Writes binary data from A to the file stream. A total of nelem objects each of size bytes is to be output; the number of such objects actually written is returned.

size_t fseek (FILE *stream, long offset, int whence)
Moves the current location within file stream by offset bytes relative to the starting point specified by whence

Function fopen (char *fname, char *mode)

- fopen takes two arguments.
  1. the first one is a string that indicates the name of the file. The string can be a file name in the current directory, or a full pathname such as /user/share/dict/words. The string can also be taken from a command-line argument.
  2. the mode.
     - → Read mode "r"
       indicates the file is to be read, and arise an error if the file doesn't exist or cannot be accessed.
     - → write mode "w"
       indicates a new file is to be created a written to. If a file already exists with that name, then it will clear its any existing content.
     - → append mode "a"
       indicates the file is to be appended to.

fscanf and fprintf

- The functions fscanf and fprintf are used to convert between internal representations and sequences of printable ASCII characters.
- ASCII-mode storage can be more expensive than internal representation. E.g. a 4-byte int variable can store the number -2,000,000,000 but ASCII string "-2000000000" requires 11-bytes

Example of using fprintf and fopen

Use fopen and fprintf to print the first two lines of each file given.

```
# include <stdio.h>
# include <stdlib.h>
# define LINE_LIMIT 2
# define SEPARATOR "==========="

/* a function that copy the first few lines of
   the file fp to stdout, and count the total
   number of line of the file fp                */
void
first_n_lines ( FILE *fp , int n) {
    int c;
    int num_of_lines ;
    while ( (c=getc (fp)) != EOF) {
        if (num_of_lines < n) {
            putchar (c);
        }                    ↳ -↑↑ to print char.
        num_of_lines += (c == '\n');
    }
    printf (" [ %d lines in total ]\n", lines);
    return;
}
```

continue →

```
int
main (int argc , char *argv [] ) {
    int fnum;
    FILE *fp;

    for (fnum = 1 ; fnum < argc ; fnum ++) {
        fprintf ( stderr , "Opening %s : ", argv [fnum] );

        if ( (fp = fopen (argv [fnum] , "r")) == NULL){
            fprintf ( stderr , " --- failed \n");
                ↓ 給到 screen
        } else {
            fprintf (stderr, "\n");
            printf (" %s %s \n", SEPARATOR , argv [fnum]);
            first_n_lines (fp, LINE_LIMIT);
            fclose (fp);
        }
    }
    return 0;
}
```

OUTPUT:

```
========== twodarray.c
/* Manipulate a two-dimensional array.
*/
[ 38 lines in total ]
========== twolines.c
/* Print the first few lines of a set of files.
*/
[45 lines in total ]
```

fread and fwrite

- In a binary file the stored form of information is exactly the same as the internal representation used in program variables. Function fread and fwrite allow variables and arrays to be written to and from files without any format conversion taking place.

Example of using fopen, fwrite, freopen, fread

```
# Create new file
    fp = fopen ("newfile.dat", "w");
FILE *fp;   assert (fp != NULL)

# copy the array's data to the file
    double A[2] = {2.2, 3.3};
    int i;

    i = fwrite (A, sizeof (*A), 2, fp);
                                ↑ number of elements
    assert (i == 2);
```

validate whether the writing/reading is successful, as if it is, the function fread/fwrite will return the total number of elements of the original array.

```
# open a file and print its stored data.
    fp = freopen ("newfile.dat", "r", fp);
    assert (fp != NULL);

    i = fread (A, sizeof (*A), 2, fp);
    assert (i == 2);
    // A 现在又等于 {2.2, 3.3}.
    fclose (fp);
```

The magic '+'

- To random access seek/read/rewrite a file, we can add a '+' after the three modes to make "r+", "w+", "a+". This indicates the the file can be operated in a mode that allow it to take input as well as print outputs.

fscanf / fwrite fread / fprintf

⇧ The only requirement is that an fseek operation must be carried out in between the change from input to output mode, or vice versa.

fseek (FILE *stream, long offset, int whence)

★例子 fseek (fp, -sizeof (staff_t), SEEK_CUR)
shifts the current location back by exactly the size of one staff_t structure.

→ So if a staff_t record had just read using freand, it can be modified in memory, and then written back to the file, overwriting the old record that was in its place. Another fseek should then be executed prior to the next fread ~~opear~~ operation.

★ Files can be opened for interleaved input and output operations. Function fseek is used to move the current location within the file. This mode of operation usually make sense if all objects in the file are of the same type.

chapter 13.

Additional operators

- "&" is an operation that operates on a bit-by-bit basis over the w bits of its two arguments.
  - if $w = 4$, then 1101 & 1001 yields 1001
    - match: 1&1 = 1
    - notmatch: 1&0 = 0
- "<<" is a left shift operator that moves its left operand left by the number of bits specified by the right operand, discarding the bits that move off the left-hand (most significant) end of the word, and introducing zero bits at the right-had (least significant) end of the word.
  - ~~if w = 4, then~~ 1102 << 2 yields 0100.
  - Left shift operation is equivalent to multiplying by a power of two. The right shift operator ">>" has the effect of dividing by a power of two.

- "|" a bitwise operator for the "or" of two integer.
  - E.g. 1101 | 1001 yields 1101
    - 1 or 1 = 1
    - 1 or 0 = 1
- "~" a bitwise complement of an integer
  - Example ~1001 = 0110
- "^" a bit wise exclusive or operator, it sets a bit in the result to 1 if the corresponding bits in the two operands differ: 1101 ^1001 gives 0100.

Unsigned variable: unsigned variable stores positive values only

Double and float.

★ Variables of type double and float are stored as an integer exponent part, and a fractional mantissa part. They are represented in three parts: (a) a sign
(b) an integer exponent
(c) a fractional mantissa.

★

Number we understand	number in binary	exponent (decimal)	Mantissa (binary)	representation (bits)
0.5	0.1	0	.1000 0000 0000	0000 1000 0000 0000
0,375	0.011	-1	.1100 0000 0000	0111 1100 0000 0000
3.1415	11.00100100…	2	.1100100 10000	0010 1100 1001 0000
-0.1	-0.0001100 1…	-3	.1100 1100 1100	1101 1100 1100 1100

Precedence hierarchy for all C operators

	Operators	Operation class	Associativity
Higest	[ ], ., ->	component selection	left to right
	++, --	post/pre increment/decrement	right to left
	!, ~, -, (type), &, *, sizeof	unary operators	right to left
	*, /, %	multiplication	left to right
	<<, >>	bit shifting	left to right
	+, -	addition	left to right
	<, >, <=, >=	comparison	left to right
	==, !=	equality	left to right
	&	bitwise and	left to right
	^	bitwise exclusive or	left to right
	|	bitwise or	left to right
	&&	and	left to right
	||	or	left to right
	?	conditional evaluation	right to left
	=, +=, *=, etc	assignment	right to left
lowest	,	expression composition	left to right

Big Oh notation

Insertion sort $\frac{(n-1)n}{4}$ $O(n^2)$

Example. 1 3 2 5 6

Compare 1 3 , 3比1大不用改

Compare 3 2 , 2比3小，改

新! 1 2 3 5 6

Compare 1 2 , 2比1大不用改

Compare 3 5 ...